

Mitali Bhatia

D20C 04

Blockchain

Experiment No 03

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Theory:

1. Blockchain Overview

Blockchain is a **distributed and decentralized ledger** that stores information in a series of linked blocks.

Each block contains:

- Transaction data
- Timestamp
- Previous block's hash
- Its own unique hash (digital fingerprint)

Once data is recorded in a blockchain, it becomes **immutable** because altering one block would require recalculating all subsequent blocks.

2. Mining

Mining is the process of:

1. Collecting pending transactions into a block.
2. Performing a computational puzzle (Proof-of-Work) to find a valid hash.
3. Adding the new block to the blockchain.

Broadcasting it to all connected peers.

Miners are rewarded with cryptocurrency for successfully mining a block.

3. Multi-Node Blockchain Network

In this lab, we simulate **three independent blockchain nodes** (5001, 5002, 5003).

Each node:

- Runs on a separate port.
- Maintains its own copy of the blockchain.
- Can connect with peers to share and validate blocks.

4. Consensus Mechanism

We use the **Longest Chain Rule**:

- If multiple versions of the chain exist, the **longest valid chain** is chosen.
- This ensures all nodes agree on a single transaction history.

5. Transactions & Mining Reward

Each transaction has:

- Sender
- Receiver
- Amount

When mining a block:

- Pending transactions are added to the block.
- A **reward transaction** is added automatically to pay the miner.

6. Chain Replacement

When /replace_chain is called:

1. Node requests chains from peers.
2. If it finds a longer and valid chain, it replaces its own.
3. This keeps the blockchain consistent across all nodes.

Tools & Libraries Used

- **Python 3.x**
- **Flask** – Web framework for API endpoints
pip install Flask
- **Requests** – For HTTP communication between nodes
pip install requests==2.18.4
- **Postman** – For testing API requests
- Python Standard Libraries:
 - datetime
 - jsonify
 - hashlib
 - uuid4
 - urlparse
 - request

Procedure:

1. Download [hadcoin_node_5001.py](#), [hadcoin_node_5002.py](#), [hadcoin_node_5003.py](#).
2. Install required packages.
3. Run each node in separate terminals.

OUTPUT:

Opened venv and installed python flask

```
PS C:\Clg\Blockchain codes\Exp 03> python -m venv venv
PS C:\Clg\Blockchain codes\Exp 03> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "c:\Clg\
Blockchain codes\Exp 03\venv\Scripts\Activate.ps1")
(venv) PS C:\Clg\Blockchain codes\Exp 03> venv\Scripts\activate
(venv) PS C:\Clg\Blockchain codes\Exp 03> pip install Flask
Collecting Flask
  Using cached flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from Flask)
  Using cached blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from Flask)
  Using cached click-8.5.0-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from Flask)
  Using cached itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from Flask)
  Using cached jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from Flask)
  Using cached markupsafe-3.0.3-cp313-cp313-win_amd64.whl.metadata (2.8 kB)
Collecting werkzeug>=3.1.0 (from Flask)
  Using cached werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Using cached flask-3.1.3-py3-none-any.whl (103 kB)
Using cached blinker-1.9.0-py3-none-any.whl (8.5 kB)
Using cached click-8.5.0-py3-none-any.whl (125 kB)
Using cached itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Using cached jinja2-3.1.6-py3-none-any.whl (134 kB)
Using cached markupsafe-3.0.3-cp313-cp313-win_amd64.whl (15 kB)
Using cached werkzeug-3.1.8-py3-none-any.whl (226 kB)
Installing collected packages: markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, Flask
Successfully installed Flask-3.1.3 blinker-1.9.0 click-8.5.0 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkz
eug-3.1.8

[notice] A new release of pip is available: 25.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

hadcoin_node_5001

```
PS C:\Clg\Blockchain codes\Exp 03\Experiments3> python hardcoin_node_5001.py
* Serving Flask app 'hardcoin_node_5001'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5001
* Running on http://10.124.178.178:5001

127.0.0.1 - - [02/Sep/2026 12:28:37] "POST /connect_node HTTP/1.1" 201 -
127.0.0.1 - - [02/Sep/2026 12:30:36] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [02/Sep/2026 12:40:58] "POST /add_transaction HTTP/1.1" 201 -
127.0.0.1 - - [02/Sep/2026 12:42:27] "POST /add_transaction HTTP/1.1" 201 -
127.0.0.1 - - [02/Sep/2026 12:44:28] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 - - [02/Sep/2026 12:44:52] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [02/Sep/2026 12:44:52] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [02/Sep/2026 12:49:32] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [02/Sep/2026 12:51:22] "GET /get_chain HTTP/1.1" 200 -
```

hadcoin_node_5002

```
PS C:\Clg\Blockchain codes\Exp 03\Experiments3> python hardcoin_node_5002.py
* Serving Flask app 'hardcoin_node_5002'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5002
* Running on http://10.124.178.178:5002
Press CTRL+C to quit
127.0.0.1 - - [02/Sep/2026 12:29:43] "POST /connect_node HTTP/1.1" 201 -
127.0.0.1 - - [02/Sep/2026 12:32:10] "POST /connect_node HTTP/1.1" 201 -
127.0.0.1 - - [02/Sep/2026 12:32:52] "POST /connect_node HTTP/1.1" 201 -
127.0.0.1 - - [02/Sep/2026 12:39:20] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [02/Sep/2026 12:50:21] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [02/Sep/2026 12:50:54] "GET /replace_chain HTTP/1.1" 200 -
127.0.0.1 - - [02/Sep/2026 12:51:22] "GET /get_chain HTTP/1.1" 200 -
```

hadcoin_node_5003

```
PS C:\Clg\Blockchain codes\Exp 03\Experiments3> python hadoin_node_5003.py
* Serving Flask app 'hadcoin_node_5003'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5003
* Running on http://10.124.178.178:5003
Press CTRL+C to quit
PS C:\Clg\Blockchain codes\Exp 03\Experiments3> 127.0.0.1 - - [02/Sep/2026 12:30:17] "GET /connect_node HTTP/1.1" 405 -
PS C:\Clg\Blockchain codes\Exp 03\Experiments3> 127.0.0.1 - - [02/Sep/2026 12:32:10] "GET /connect_node HTTP/1.1" 405 -
PS C:\Clg\Blockchain codes\Exp 03\Experiments3> 127.0.0.1 - - [02/Sep/2026 12:33:55] "GET /connect_node HTTP/1.1" 405 -
PS C:\Clg\Blockchain codes\Exp 03\Experiments3> 127.0.0.1 - - [02/Sep/2026 12:34:20] "GET /connect_node HTTP/1.1" 405 -
PS C:\Clg\Blockchain codes\Exp 03\Experiments3> 127.0.0.1 - - [02/Sep/2026 12:34:22] "POST /connect_node HTTP/1.1" 201 -
PS C:\Clg\Blockchain codes\Exp 03\Experiments3> 127.0.0.1 - - [02/Sep/2026 12:39:59] "GET /get_chain HTTP/1.1" 200 -
PS C:\Clg\Blockchain codes\Exp 03\Experiments3> 127.0.0.1 - - [02/Sep/2026 12:50:26] "GET /get_chain HTTP/1.1" 200 -
PS C:\Clg\Blockchain codes\Exp 03\Experiments3> 127.0.0.1 - - [02/Sep/2026 12:50:54] "GET /get_chain HTTP/1.1" 200 -
PS C:\Clg\Blockchain codes\Exp 03\Experiments3> 127.0.0.1 - - [02/Sep/2026 12:51:22] "GET /replace_chain HTTP/1.1" 200 -
```

Connecting the Blockchain Nodes - POST /connect_node.

The screenshot shows a REST client interface with the following details:

- URL:** `http://127.0.0.1:5001/connect_node`
- Method:** POST
- Body (raw):**

```
{
  "nodes": [
    "http://127.0.0.1:5002",
    "http://127.0.0.1:5003"
  ]
}
```
- Status:** 201 CREATED
- Response (JSON):**

```
{
  "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
  "total_nodes": [
    "127.0.0.1:5003",
    "127.0.0.1:5002"
  ]
}
```

http://127.0.0.1:5002/connect_node

POST http://127.0.0.1:5002/connect_node Send

Docs Params Authorization Headers (9) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "nodes": [
3     "http://127.0.0.1:5001",
4     "http://127.0.0.1:5003"
5   ]
6 }
```

Body Cookies Headers (5) Test Results 201 CREATED 6 ms 325 B

{ } JSON Preview Visualize

```
1 {
2   "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
3   "total_nodes": [
4     "127.0.0.1:5001",
5     "127.0.0.1:5003"
6   ]
7 }
8
```

http://127.0.0.1:5003/connect_node

POST http://127.0.0.1:5003/connect_node Send

Docs Params Authorization Headers (9) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "nodes": [
3     "http://127.0.0.1:5001",
4     "http://127.0.0.1:5002"
5   ]
6 }
```

Body Cookies Headers (5) Test Results 201 CREATED 5 ms 325 B

{ } JSON Preview Visualize

```
1 {
2   "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
3   "total_nodes": [
4     "127.0.0.1:5002",
5     "127.0.0.1:5001"
6   ]
7 }
8
```

Added Transactions - POST /add_transaction

The screenshot shows a REST client interface with the URL `http://127.0.0.1:5001/add_transaction`. The method is set to **POST**. The request body is a JSON object: `{ "sender": "Vanshika", "receiver": "Hitesh", "amount": 15 }`. The response status is **201 CREATED** with a response time of 7 ms and a size of 226 B. The response body is a JSON object: `{ "message": "This transaction will be added to Block 2" }`.

```
1 {
2   "sender": "Vanshika",
3   "receiver": "Hitesh",
4   "amount": 15
5 }
```

```
1 {
2   "message": "This transaction will be added to Block 2"
3 }
4
```

Mined the Blockchain - GET /mine_block

The screenshot shows a REST client interface with the URL `http://127.0.0.1:5001/mine_block`. The method is set to **GET**. The response status is **200 OK** with a response time of 7 ms and a size of 226 B. The response body is a JSON object containing transaction details.

```
{
  "index": 2,
  "message": "Congratulations, you just mined a block!",
  "previous_hash": "6c2b2e39de6769815124a4d07fd5952d1c370a910e432237f3644ba551ae3ecc",
  "proof": 533,
  "timestamp": "2026-09-02 12:41:34.561114",
  "transactions": [
    {
      "amount": 10,
      "receiver": "Prishita",
      "sender": "Mitali"
    },
    {
      "amount": 20,
      "receiver": "Vanshika",
      "sender": "Prishita"
    },
    {
      "amount": 15,
      "receiver": "Hitesh",
      "sender": "Vanshika"
    }
  ]
}
```

```
    "amount": 1,  
    "receiver": "Richard",  
    "sender": "1859d0403d3645589682f465de72d753"  
  }  
]  
}
```

Fetching the Blockchain - GET /get_chain

http://127.0.0.1:5001/get_chain

GET http://127.0.0.1:5001/get_chain

200 OK • 4 ms • 1.21 KB

```
{  
  "chain": [  
    {  
      "index": 1,  
      "previous_hash": "0",  
      "proof": 1,  
      "timestamp": "2026-09-02 12:28:29.591277",  
      "transactions": []  
    },  
    {  
      "index": 2,  
      "previous_hash": "6c2b2e39de6769815124a4d07fd5952d1c370a910e432237f3644ba551ae3ecc",  
      "proof": 533,  
      "timestamp": "2026-09-02 12:41:34.561114",  
      "transactions": [  
        {  
          "amount": 10,  
          "receiver": "Prishita",  
          "sender": "Mitali"  
        }  
      ]  
    }  
  ]  
}
```

Different Blockchain Lengths

Chain length of http://127.0.0.1:5001

http://127.0.0.1:5001/get_chain

GET http://127.0.0.1:5001/get_chain

Send

Docs Params Authorization Headers (7) Body Scripts Settings Cookies

Query Params

Key	Value	Description

Body Cookies Headers (5) Test Results

200 OK • 4 ms • 1.21 KB

{ } JSON Preview Visualize

```
52      "sender": "1859d0403d3645589682f465de72d753"
53    }
54  ],
55  {
56    "index": 4,
57    "previous_hash": "47b3c108a7d7f4eb3672d1222f9bfb3edaa09f7d7453d2b96af5e9ab1cf2ab6",
58    "proof": 21391,
59    "timestamp": "2026-09-02 12:53:15.552613",
60    "transactions": [
61      {
62        "amount": 1,
63        "receiver": "Richard",
64        "sender": "1859d0403d3645589682f465de72d753"
65      }
66    ]
67  },
68  ],
69  "length": 4
70 }
71
72
```

Chain length of http://127.0.0.1:5003

http://127.0.0.1:5002/get_chain

GET http://127.0.0.1:5002/get_chain

Send

Docs Params Authorization Headers (7) Body Scripts Settings Cookies

Query Params

Key	Value	Description

Body Cookies Headers (5) Test Results

200 OK • 4 ms • 697 B

{ } JSON Preview Visualize

```
31      "amount": 1,
32      "receiver": "Richard",
33      "sender": "1859d0403d3645589682f465de72d753"
34    }
35  ],
36  ],
37  "length": 2
38 }
39
40
41
```

Add transaction

The screenshot shows a REST client interface with the URL `http://127.0.0.1:5001/add_transaction`. The method is **POST**. The request body is raw JSON:

```
1 {
2   "sender": "Mitali",
3   "receiver": "Prishita",
4   "amount": 100
5 }
```

The response status is **201 CREATED** with a response time of 5 ms and a body size of 226 B. The response body is JSON:

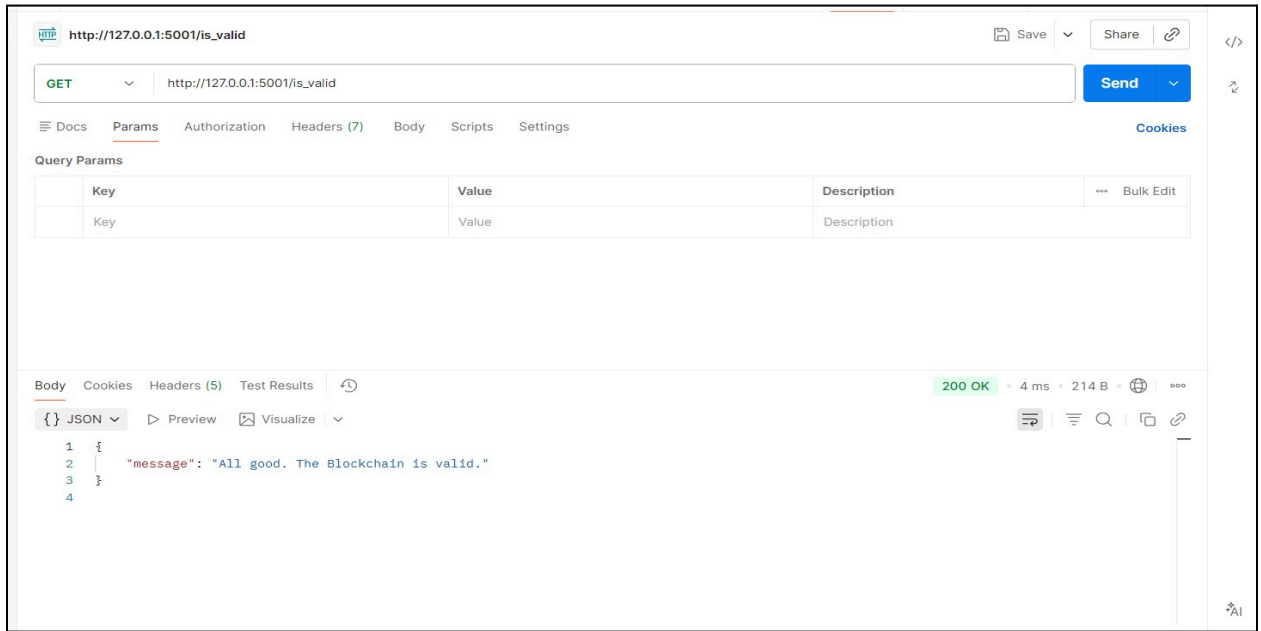
```
1 {
2   "message": "This transaction will be added to Block 3"
3 }
4
```

GET-/mine_block

The screenshot shows a REST client interface with the URL `http://127.0.0.1:5001/mine_block`. The method is **GET**. The response status is **200 OK** with a response time of 58 ms and a body size of 464 B. The response body is JSON:

```
1 {
2   "index": 4,
3   "message": "Congratulations, you just mined a block!",
4   "previous_hash": "47b3c108a7d7f4eb3672d1222f9bfb3edaa09f7d7453d2b96af5e9ab1cf2ab6",
5   "proof": 21391,
6   "timestamp": "2026-09-02 12:53:15.552613",
7   "transactions": [
8     {
9       "amount": 1,
10      "receiver": "Richard",
11      "sender": "1859d0483d3645589682f465de72d753"
12    }
13  ]
14 }
15
```

Validating the Blockchain - GET /is_valid



http://127.0.0.1:5001/is_valid

GET http://127.0.0.1:5001/is_valid

Send

Docs Params Authorization Headers (7) Body Scripts Settings

Query Params

Key	Value	Description
Key	Value	Description

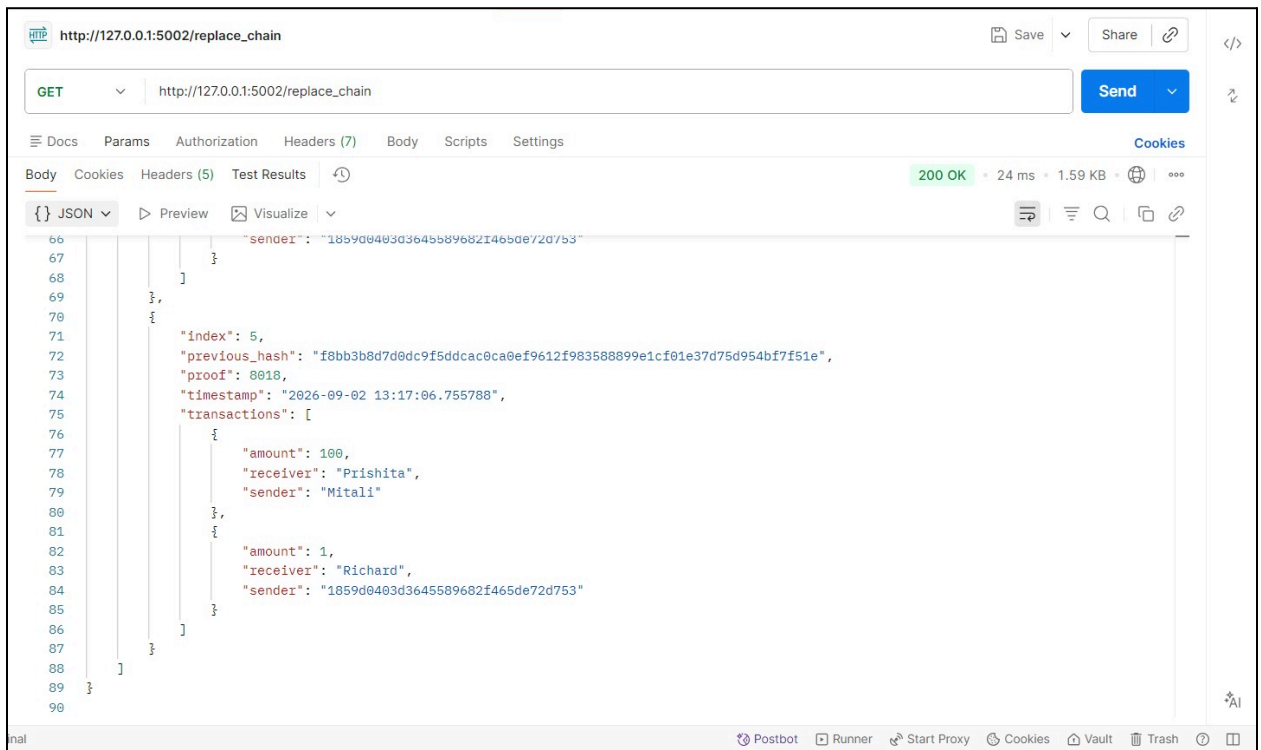
Body Cookies Headers (5) Test Results

200 OK • 4 ms • 214 B

{ } JSON Preview Visualize

```
1 {
2   "message": "All good. The Blockchain is valid."
3 }
4
```

Replacing the Chain on Node 5002 - GET /replace_chain



http://127.0.0.1:5002/replace_chain

GET http://127.0.0.1:5002/replace_chain

Send

Docs Params Authorization Headers (7) Body Scripts Settings

Body Cookies Headers (5) Test Results

200 OK • 24 ms • 1.59 KB

{ } JSON Preview Visualize

```
66 {
67   "sender": "1859d0403d3645589682f465de72d753"
68 }
69 },
70 {
71   "index": 5,
72   "previous_hash": "f8bb3b8d7d0dc9f5ddcac0ca0ef9612f983588899e1cf01e37d75d954bf7f51e",
73   "proof": 8018,
74   "timestamp": "2026-09-02 13:17:06.755788",
75   "transactions": [
76     {
77       "amount": 100,
78       "receiver": "Prishita",
79       "sender": "Mitali"
80     },
81     {
82       "amount": 1,
83       "receiver": "Richard",
84       "sender": "1859d0403d3645589682f465de72d753"
85     }
86   ]
87 }
88 ]
89 }
90
```

Replacing the Chain on Node 5003 - GET /replace_chain

REST client interface showing a GET request to `http://127.0.0.1:5003/replace_chain`. The response is a JSON object representing a block with index 5, containing two transactions.

```
66 {
67   "sender": "1859d0403d3645589682f465de72d753"
68 }
69 },
70 {
71   "index": 5,
72   "previous_hash": "f8bb3b8d7d0dc9f5ddcac0ca0ef9612f983588899e1cf01e37d75d954bf7f51e",
73   "proof": 8018,
74   "timestamp": "2026-09-02 13:17:06.755788",
75   "transactions": [
76     {
77       "amount": 100,
78       "receiver": "Prishita",
79       "sender": "Mitali"
80     },
81     {
82       "amount": 1,
83       "receiver": "Richard",
84       "sender": "1859d0403d3645589682f465de72d753"
85     }
86   ]
87 }
88 ]
89 }
90 }
```

Final nodes:

`http://127.0.0.1:5001`

REST client interface showing a GET request to `http://127.0.0.1:5001/get_chain`. The response is a JSON object representing the entire chain with 5 blocks.

```
70 {
71   "index": 5,
72   "previous_hash": "f8bb3b8d7d0dc9f5ddcac0ca0ef9612f983588899e1cf01e37d75d954bf7f51e",
73   "proof": 8018,
74   "timestamp": "2026-09-02 13:17:06.755788",
75   "transactions": [
76     {
77       "amount": 100,
78       "receiver": "Prishita",
79       "sender": "Mitali"
80     },
81     {
82       "amount": 1,
83       "receiver": "Richard",
84       "sender": "1859d0403d3645589682f465de72d753"
85     }
86   ]
87 },
88 "length": 5
89 }
90 }
```

http://127.0.0.1:5002

http://127.0.0.1:5002/get_chain

GET http://127.0.0.1:5002/get_chain

200 OK • 6 ms • 1.51 KB

```
{
  "index": 5,
  "previous_hash": "f8bb3b8d7d0dc9f5ddcac0ca0ef9612f983588899e1cf01e37d75d954bf7f51e",
  "proof": 8018,
  "timestamp": "2026-09-02 13:17:06.755788",
  "transactions": [
    {
      "amount": 100,
      "receiver": "Prishita",
      "sender": "Mitali"
    },
    {
      "amount": 1,
      "receiver": "Richard",
      "sender": "1859d0403d3645589682f465de72d753"
    }
  ]
},
"length": 5
]
```

http://127.0.0.1:5003

http://127.0.0.1:5003/get_chain

GET http://127.0.0.1:5003/get_chain

200 OK • 5 ms • 1.51 KB

```
{
  "index": 5,
  "previous_hash": "f8bb3b8d7d0dc9f5ddcac0ca0ef9612f983588899e1cf01e37d75d954bf7f51e",
  "proof": 8018,
  "timestamp": "2026-09-02 13:17:06.755788",
  "transactions": [
    {
      "amount": 100,
      "receiver": "Prishita",
      "sender": "Mitali"
    },
    {
      "amount": 1,
      "receiver": "Richard",
      "sender": "1859d0403d3645589682f465de72d753"
    }
  ]
},
"length": 5
]
```

Conclusion:

A cryptocurrency was successfully created using Python and implemented using three blockchain nodes. Transactions were added to the blockchain and blocks were mined using the Proof-of-Work mechanism. The pending transactions were removed after being added to a block. Finally, all nodes were synchronized using the Longest Chain Rule, demonstrating blockchain mining, transactions, validation, and network synchronization.